ICS 604
Applied Data Science

Department of Information and Computer Sciences
University of Hawai`i at Mānoa

Kyungim Baek

Announcements

- Grades for homework assignment 1 have been posted.
  - Feedback available on Lamaku.

Lecture 10

- Simulation-Based Inference for a Generative Process (cont'd.)
  - Working with Non-Uniform Probabilities
- Probability Distributions
  - Random Variables
  - Binomial Distribution

Working with non-uniform probabilities

- Easy to simulate problems with uniform probabilities.
  - We know how to generate events for an experiment whose sample space is uniformly distributed.
    - Use `random.choice()` and pass it a list representing the sample space.
- What about non-uniform probabilities?

Working with non-uniform probabilities: Example

You are the CEO of your startup, and you are preparing for a funding round for your new software. You want to show that users like some newly added functionality.

- Testing results on 30 users is the minimum number you are willing to report your VCs (investors).
- The probability of a person responding to your request to volunteer to test your software is 0.03.
  - On average, out of ~100 people who receive your brochure by email, three will volunteer to test your software.
- You don't have a marketing database of potential users, and a company is charging you per email sent.

Working with non-uniform probabilities: Example (cont'd.)

- Package that includes 1,200 emails sent is within your budget, but what is the probability it will lead to at least 30 responses?
  - What is the probability of having at least 30 people volunteering to test your software if you send the 1,200 emails to people?
- Simulation strategy:
  1. We encode those who volunteer as 1 and those who don't as 0.
  2. We generate a vector that represents your population.
     - Your population reflects the proportions of participants and non-participants observed in the population.
  3. Use `np.random.choice()` to sample 1,200 people from the population and estimate the probability that at least 30 will participate.

```
# 1. Encode those who volunteer as 1 and those who don't as 0, and
# 2. Generate a population vector that represents the ratio of users
#    likely to volunteer.
#    In 100 people, we have (voluteer probability = 0.03):

population = [0] * 97  + [1] * 3
```

97 0s 3 1s = [0, 0, ... , 0, 1, 1, 1]

```
# The code below can be easily written using numpy.random.choice() method

def get_random_sample(population, nb_samples):
    nb_volunteers = 0
    for _ in range(nb_samples):
        nb_volunteers += random.choice(population)
    return nb_volunteers
```

np.random.choice (population, size = 1200)

```
get_random_sample(population, 1200)
```

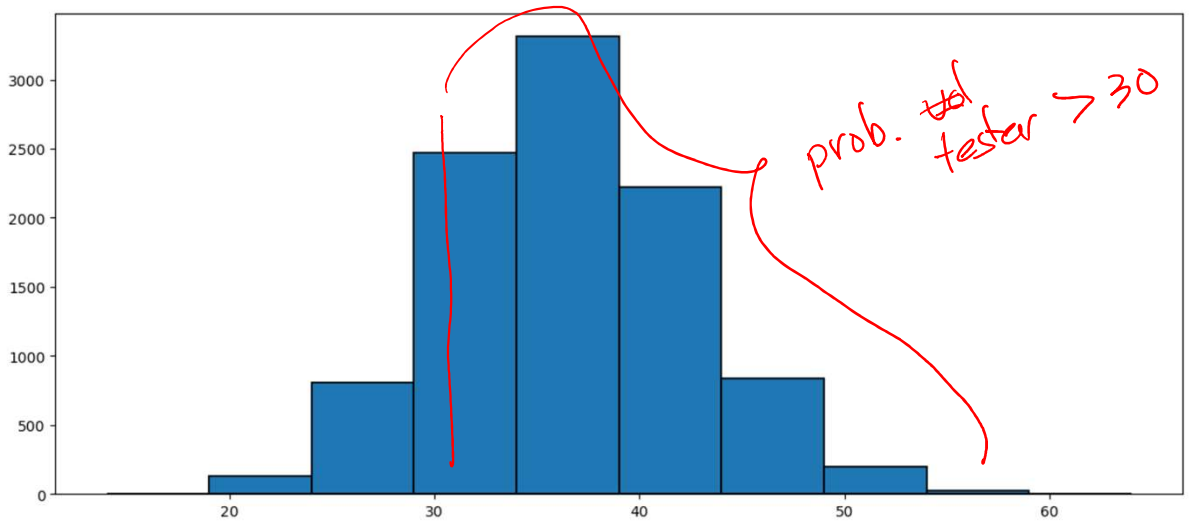
```
33
```

```
# Recall that the probability is the long-run proportion with which
# a certain event will occur.
# 3. Repeat the experiment and count the number of times you have
# at least 30 volunteers, i.e., satisfied with the outcome.

nb_satisfied = 0
nb_trials = 10_000
nb_volunteers_per_exp = []
for i in range(nb_trials):
    nb_volunteers = get_random_sample(population, 1200)
    nb_volunteers_per_exp.append(nb_volunteers)
```

```
# 3. Probability that at least 30 will participate
nb_satisfied = sum(np.array(nb_volunteers_per_exp) > 30)
print(nb_satisfied/nb_trials)
```

```
0.8294
```

```
plt.figure(figsize=(14, 6))
plt.hist(nb_volunteers_per_exp, edgecolor='black', linewidth=1.2)
plt.show()
```



Probability distributions

- There are much easier ways to compute probabilities in this and other popular (canonical) problems.
- Numpy and SciPy offer parameterized functions which, given a parameter, can easily compute the probability of any outcome in the sample space.
  - Those are called **probability distributions**.

Probability models

- A way to reason about a probabilistic system from various perspectives.
- E.g., you want to run an ad campaign through an agency. You may be interested in:
  - The average number of new clients that we should expect after the ad campaign?
  - Determine the amount of variability you should expect to see in the system.
    - How large is that number? E.g., a value 300-400 is more informative than 100-1,000.
  - Compute probability of events.
    - What is the probability of having more than X new clients after a given ad campaign?
- Probabilistic models built around three core components:
  - **Random variables**: What is uncertain in the system
  - **Probability distributions**: How uncertainty behaves
    - Describes how likely different outcomes are for the random variables.
  - **Parameters**: Numerical values that specify a distribution

Probability models: An example

- E.g., you want to run an ad campaign through an agency and are interested in:
  1. Average number of new clients after an ad campaign
  2. Variability around that number
  3. The probability of having at least X new participants
- We can repeat the ad campaign a large number of times and answer each of these questions.

```
nb_new_participants = [500, 800, 860, 820, 840, 760, 430,...]
```

Probability models: An example (cont'd.)

1. Average number of new clients after an ad campaign

```
np.mean(nb_new_participants)
```

2. Variability around that number

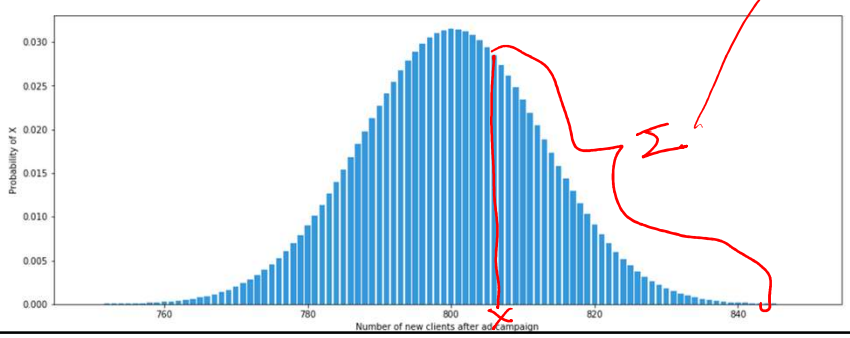
```
np.var(nb_new_participants)
```

3. The probability of having at least X new participants

```
sum(array(nb_new_participants) > X) / len(nb_new_participants)
```

Probability models: An example (cont'd.)

- The same information is contained in the visualization of the data.
  - What does every single bar mean?
  - How can you get an idea of the mean, variance, and probability of at least X participants?



Probability distributions

- At the heart of probability models are **probability distributions** and **random variables**.
- Consider the loaded die with the following probabilities:

```
p(1) = 0.3/6
p(2) = 0.7/6
p(3) =   2/6
p(4) = 0.5/6
p(5) = 0.2/6
p(6) = 2.3/6
```

Probability distributions (cont'd.)

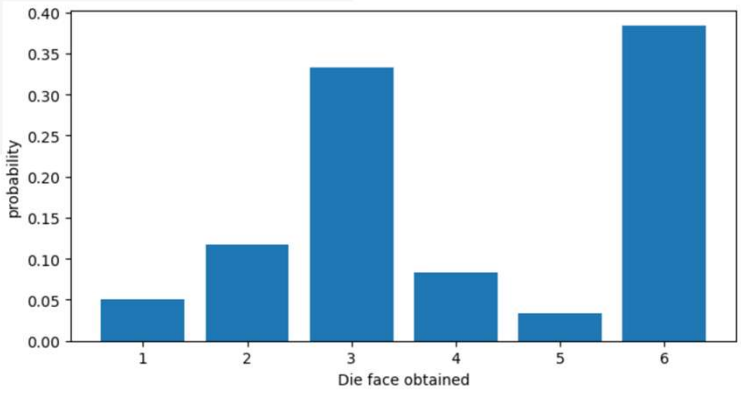
- We can think of probabilities in the previous slide as the values returned by a function that:
  - takes as input a value in {1, 2, 3, 4, 5, 6}, and
  - returns the relevant probability from the list [0.3/6, 0.7/6, 2/6, 0.5/6, 0.2/6, 2.3/6].
- In general, a probability distribution can refer to:
  - A probability measure on a sample space *S*.
  - The distribution of a random variable *X*, i.e., a probability measure on the value space of X.

{HH, HT, TH, TT}

# of Heads $X \in \{0, 1, 2\}$

```python
def loaded_die_distribution(face):
    try:
        dist = {1: 0.3/6, 2: 0.7/6, 3: 2/6, 4: 0.5/6, 5: 0.2/6, 6: 2.3/6}
        return dist[face]
    except:
        print(f"{face} is not a valid die face")
        raise
```

```python
sample_space = list(range(1, 7))
p_x = [loaded_die_distribution(f) for f in sample_space]

plt.figure(figsize=(8, 4))
plt.bar(sample_space, p_x)

plt.xlabel('Die face obtained')
plt.ylabel('probability');
```



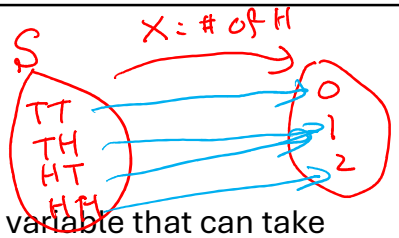
Sampling from the biased die

- Given our probability distribution, we are four times more likely to draw a 3 than we are to draw a 4.
- To sample die rolls, we need to encode our probability distribution to reflect the bias in our die.
  - This can be easily done using the `choice()` function in `np.random`.

```python
for _ in range(20):
    print(np.random.choice([1, 2, 3, 4, 5, 6],
                           p=[0.3/6, 0.7/6, 2/6, 0.5/6, 0.2/6, 2.3/6]
                          ),
          end=' ')
```

```
6 6 3 3 2 3 6 1 3 3 4 3 1 6 6 4 6 6 2 2
```

Random variables



- Using CS thinking, a random variable is merely a variable that can take values that are variable (random).
- In probabilistic thinking, a random variable is a variable that can take any value from the sample space of the experiment as determined by the probability distribution.
  - The value assigned to the variable depends on a random process.

```
# randomly select a value according to a probability distribution
dieRollVal = np.random.choice([1, 2, 3, 4, 5, 6], p=[0.3/6, 0.7/6, 2/6, 0.5/6, 0.2/6, 2.3/6]);
```

- Formally, a random variable is a function from the sample space of an experiment to the set of real numbers. That is, a random variable assigns a real number to each possible outcome.

Random variables (cont'd.)

- In a non-probabilistic program, you can trace the value contained in a variable.
- In a probabilistic program, you cannot since the value is determined probabilistically.
  - We don't know what that variable will contain prior to executing the code.
  - We can, however, reason about what the value will most likely contain.

Random variables (cont'd.)

- Take for instance the variable `roll_random_var` in the code below:

```
for i in range(10):
    roll_random_var = np.random.choice([1, 2, 3, 4, 5, 6],
                                       p=[0.3/6, 0.7/6, 2/6, 0.5/6, 0.4/6, 2.1/6])
    print (f"roll_random_var contains the value {roll_random_var}")
```

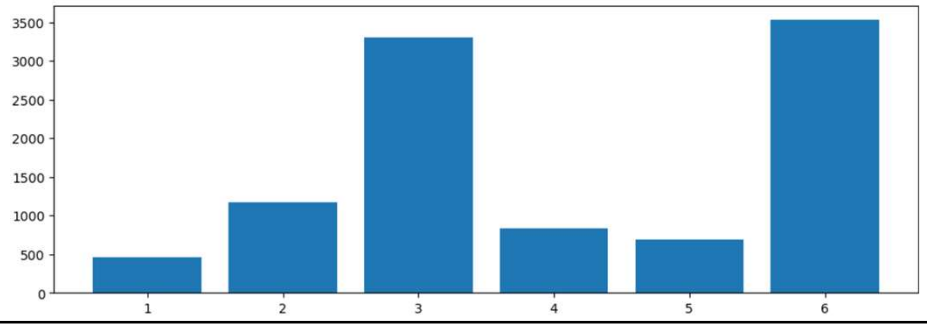
- The variable `roll_random_var` is a random variable.
  - The values assigned to it are determined probabilistically using a probability distribution of the biased die.

```
for i in range(3):
    roll_random_var = np.random.choice([1, 2, 3, 4, 5, 6],
                                       p=[0.3/6, 0.7/6, 2/6, 0.5/6, 0.4/6, 2.1/6])
    print(f"roll_random_var contains the value {roll_random_var}")
```

```
roll_random_var contains the value 3
roll_random_var contains the value 6
roll_random_var contains the value 5
```

```
from collections import Counter

roll_random_var_list = np.random.choice([1, 2, 3, 4, 5, 6], size=10000,
                                        p=[0.3/6, 0.7/6, 2/6, 0.5/6, 0.4/6, 2.1/6])

counts = Counter(roll_random_var_list)
sorted_counts = {int(k): v for k, v in sorted(counts.items())}
print(sorted_counts)

plt.figure(figsize=(12, 4))
plt.bar(sorted_counts.keys(), sorted_counts.values())
plt.show()
```

```
{1: 460, 2: 1205, 3: 3255, 4: 851, 5: 724, 6: 3505}
```



Two types of random variables

- Discrete (finite or countably infinite)
  - Take as input finite or countably finite integer values.
  - Examples: number of accidents on H1 on any given day, number of customers at lunchtime at Starbucks on campus, number of siblings a student at UH has, etc.
- Continuous (uncountably infinite)
  - Take as input uncountably infinite float values.
  - Examples: height, weight, time, etc.

Two types of random variables (cont'd.)

- Discrete and continuous random variables have commonly occurring distributions.
  - These distributions are simply functions – just like the one encoded previously – that give the probability or probability density for each element in the sample space.
- Perhaps two of the most popular distributions are the binomial (discrete) and the normal (or Gaussian) (continuous).

Binomial distribution (The "Hello World" of probability distributions)

- A binomial distribution is simply the probability of number of SUCCESS outcomes in an experiment that is repeated multiple times.
  - $\boldsymbol{m}$ successes in an experiment of $\boldsymbol{n}$ **fixed, independent** trials.
- Each trial has a probability of success $\boldsymbol{p}$, and a probability of failure $\mathbf{1} - \boldsymbol{p}$ (heads and tails; 0 and 1; etc.).

- binary outcome
- fixed # of trials = n
- independent trial
- prob. of success outcome = p

Examples of the binomial distribution

1. You flip a fair coin 100 times. What is the probability of observing 27 heads?
2. Given an ad clickthrough probability of $p = 0.1$, what is the probability of having 100 clicks if we show the ad 1,000 times?
3. Given a disease prevalence probability of $p = 0.05$, what is the probability of observing the disease in at least 100 individuals if we survey 5,000 randomly selected individuals from the population?

What are p, n, and m in each of these experiments?

Question

- Which of these situations can be modeled using a binomial distribution?
  a) Rolling a die 10 times and counting sixes
  b) Counting how many customers enter a store between 2–3pm
  c) Testing 20 light bulbs and counting defectives
  d) Flipping a coin until the first head appears

- binary outcome
- fixed n trials
- independent trials
- p: prob. of success

Binomial distribution (cont'd.)

- Given the number of trials and the probability of success p, the probability of m successes:

$$p(X = m) = \binom{n}{m} p^m \times (1-p)^{n-m}$$

- This equation is the probability function for a random variable *X* distributed according to a binomial distribution.
  - Think of it as a convenient function to assign probabilities to any value of the random variable.

Binomial distribution (cont'd.)

- We can easily model (reason about) a problem that fits the binomial since we have access to the distribution.
  - E.g., assume that after downloading your app, the probability of subscribing to a monthly plan in your app is p = 0.05.
    - If, on average, 10k users download your app each month, how many (on average) subscribed users will you have after one year?
    - What is the probability of having at least 3,000 subscribed users after one year?
  - You can use this simple model to derive various valuable insights about your app.
    - E.g., total revenue after one year, the number of servers you will need to support your user base, the number of customer service employees to support your customers, etc.

```python
from scipy.special import comb

def binomial_probability(n, m, p):
    """
    Calculate the binomial probability.

    :param n: Number of trials
    :param m: Number of successful outcomes
    :param p: Probability of success in each trial
    :return: Binomial probability
    """
    probability = comb(n, m) * (p ** m) * ((1 - p) ** (n - m))
    return probability

n = 10  # Total number of trials
m = 3   # Number of successes
p = 0.5 # Probability of success in each trial

print(binomial_probability(n, m, p))
```

$\binom{n}{m} \times p^m \times (1-p)^{n-m}$

```
0.1171875
```

Binomial distribution (cont'd.)

- Once we know the parameter of the distribution (e.g., here $p = 0.05$), it's easy to answer the questions listed in the previous slide.
  - This may be trivial in some cases and challenging in others.
- Questions:
  - How did we get the $p = 0.05$?
  - How sure are we of that value?
  - How likely is it to be $p = 0.04$ or $p = 0.06$ instead?

Questions

You launch a new feature in your app and assume each user independently adopts it with probability 0.05. You have 1000 users.

- What's your intuitive guess for the *most likely* number of adopters?
- If you actually observe 150 adopters, what conclusions (if any) would you tentatively draw about your assumed 0.05 probability? What about the model you assumed?

Probability distributions for discrete random variables

- A probability distribution for a discrete random variable is called probability mass function (**pmf**).
  - `pmf`s take a discrete random variable X and assign a probability to each value of its sample space.
- These functions allow us to compute probabilities for events seamlessly.
  - A byproduct is that we can also derive expected values, probabilities for subspaces, and use them to sample from that distribution, etc.

pmf in Python

- Programming languages and data analysis software commonly support popular pmfs.

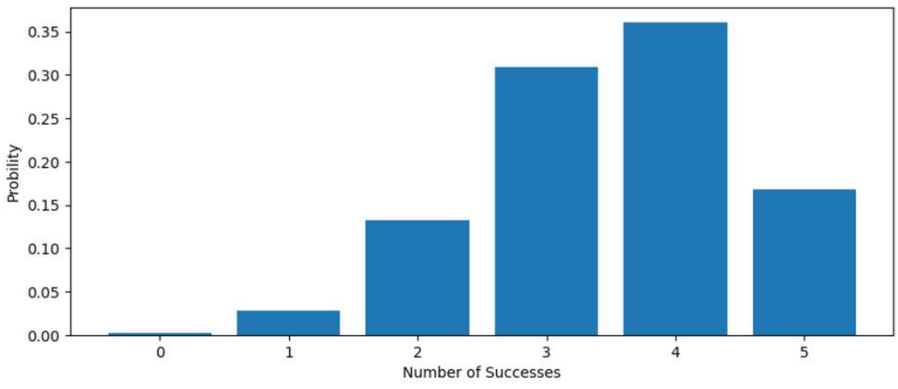
```
from scipy.stats import binom

n = 5
p = 0.7
print(binom.pmf(1, n, p))
```

```
0.02834999999999999
```

- The common graphical representation for a pmf is a histogram.
- The x-axis contains observed values.
- The y-axis shows the probability for the corresponding x values.

```
x = range(n + 1)
p_x = [binom.pmf(val, n, p) for val in x]
print(sum(p_x))

plt.figure(figsize=(10, 4))
plt.bar(x, p_x)
plt.xlabel("Number of Successes")
plt.ylabel("Probility");
```

```
1.0
```



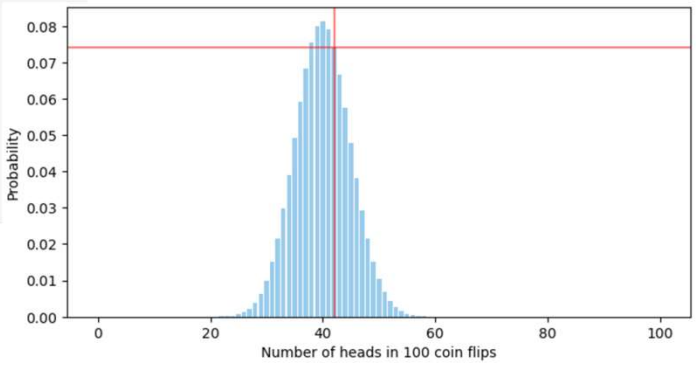
Example

- Given a biased coin that comes up heads with a probability (p = 0.4).
- If you flip the coin 100 times, what is the probability of observing 42 heads?

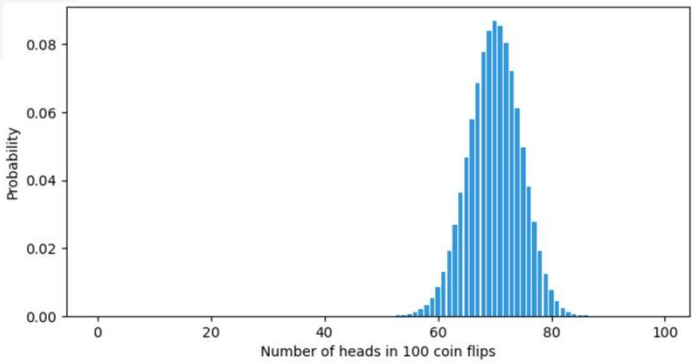
```
from scipy.stats import binom

n = 100
p = 0.4
print(binom.pmf(42, n, p))
```

```
0.07420719403438258
```

```
n = 100
p = 0.4

x = range(n + 1)  # our x-axis
p_x = [binom.pmf(val, n, p) for val in x]  # our y-axis

plt.figure(figsize=(8, 4))
plt.bar(x, p_x, color="#3498db", alpha=0.5)

# Add the horizontal and vertical bars
x_val = 42
y_val = binom.pmf(42, n, p)

plt.axvline(x_val, color='r', alpha=0.5)
plt.axhline(y_val, color='r', alpha=0.5)
plt.xlabel(f'Number of heads in {n} coin flips')
plt.ylabel('Probability')
plt.show()
```

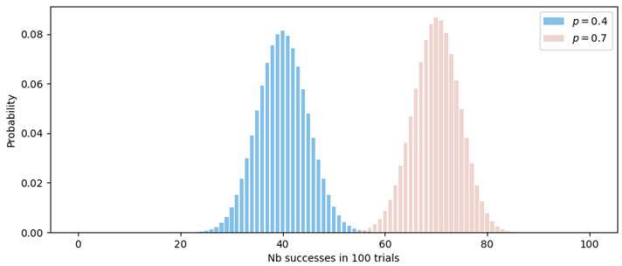


```python
n = 100
p = 0.7

x = range(n + 1)
p_x = binom.pmf(x, n, p)

plt.figure(figsize=(8, 4))
plt.bar(x, p_x, color="#3498db")
plt.xlabel(f'Number of heads in {n} coin flips')
plt.ylabel('Probability')
plt.show()
```



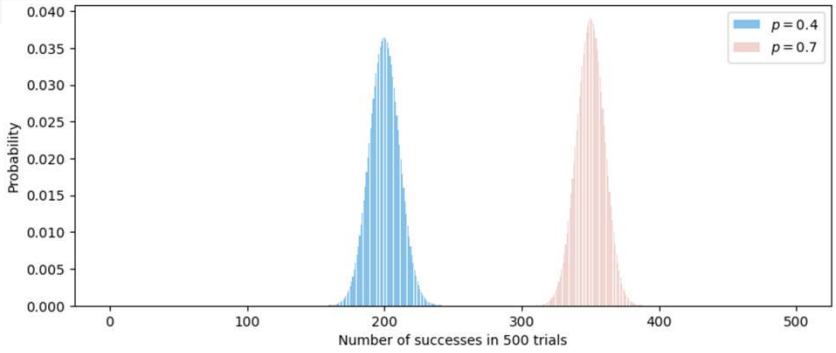
Comparing distributions

- From the two previous examples, it is evident that the two distributions differ.

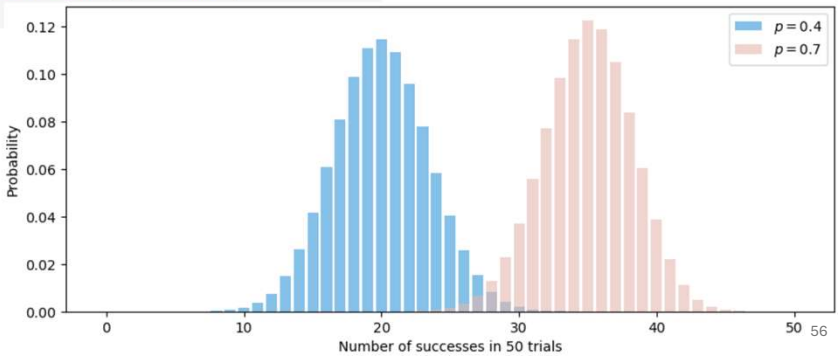


- Binomial distributions parameterized with different values of *n* and/or *p* will produce different `pmf`s.
- Therefore, it is important to describe a pmf's parameters when describing it.
  - Modifying these parameters results in a different probability distribution.

```
n = 500
p = 0.4
x = range(n + 1) # our x-axis
p_x = [binom.pmf(val, n, p) for val in x]  # our y-axis
plt.figure(figsize=(10, 4))
plt.bar(x, p_x, color="#3498db", alpha=0.6, label="$p=0.4$")

p = 0.7
p_x =  [binom.pmf(val, n, p) for val in x]  # our y-axis
plt.bar(x, p_x, color="#e6b8af", alpha=0.6, label="$p=0.7$")
plt.legend()
plt.xlabel(f"Number of successes in {n} trials")
plt.ylabel("Probability")
```




```
n = 50
p = 0.4
x = range(n + 1) # our x-axis
p_x = binom.pmf(x, n, p)   # our y-axis
plt.figure(figsize=(10, 4))
plt.bar(x, p_x, color="#3498db", alpha=0.6, label="$p=0.4$")

p = 0.7
p_x = binom.pmf(x, n, p)  # our y-axis
plt.bar(x, p_x, color="#e6b8af", alpha=0.6, label="$p=0.7$")
plt.legend()
plt.xlabel(f"Number of successes in {n} trials")
plt.ylabel("Probability")
```



56

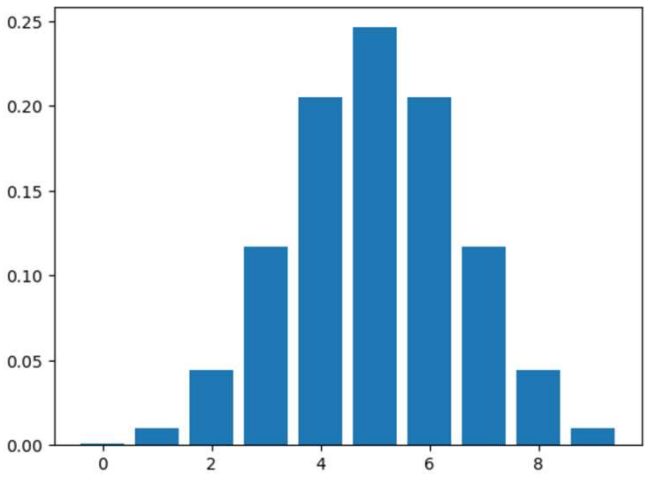
Binomial distribution summary: X, n, p, and the pmf

- The number of observed successes is said to be a random variable X, distributed as a binomial distribution.
- We write the above statement as:

$$X \sim Bionomial(n, p)$$

- This simply means that:
  - The random variable X takes on values in the sample space describes as a binomial distribution.
    - The sample space here is countably finite and range from 0 to n.
  - The parameters of the binomial probability distributions are n and p.

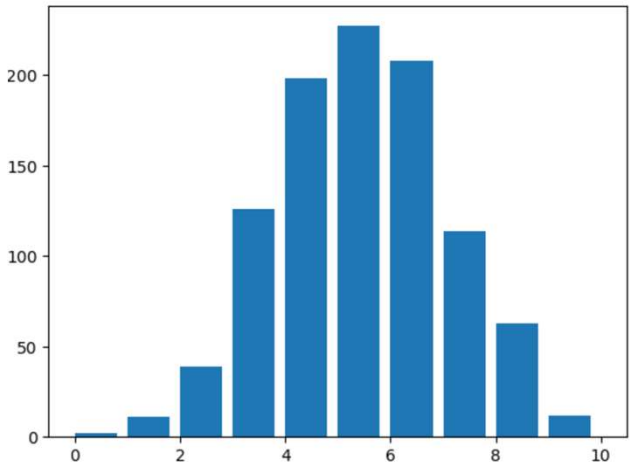
```
x = range(0, 10)
p_x = binom.pmf(x, n=10, p=0.5)
plt.bar(x, p_x)
```



```
x = binom.rvs(n=10, p=0.5, size=1000)
x
```

```
array([ 3,  7,  7,  6,  7,  4,  5,  2,  3,  6,  2,  5,  6,  3,  5,  4,  6,
        4,  6,  5,  7,  5,  3,  5,  6,  7,  8,  7,  5,  5,  2,  6,  6,  3,
       ⋮
```

```
plt.hist(x, width=0.8)
```



Class participation question

Post your answers to the question below in Slack before the next class. Please prefix your post with: "Binomial 2/23"

- **Questions**:

  The number of side effects in a clinical trial out of 100 patients can be modeled using a binomial distribution.

  1. Explain how this scenario satisfies the conditions required for a binomial model: fixed number of trials, independent trials, binary outcomes, and constant probability of success
  2. What real-life questions could be answered with this model?